フォーマットフォルダツリー UI仕様書

1. 概要

1.1 コンポーネントの目的

フォーマットフォルダツリーは、部門配下のフォーマットフォルダ一覧を縦型リスト形式で表示するコンポーネントです。各フォーマットフォルダの基本情報（名前、ファイルタイプ、ファイル数、更新日）を表示し、ユーザーがフォーマットを選択できるようにします。

1.2 使用場面

- **表示場所**: フォーマット管理画面の左側パネル（画面幅の約33%）
- **主な用途**:
  - フォーマットフォルダの一覧表示
  - フォーマットフォルダの選択
  - ソート機能（名前/カテゴリ/更新日）

1.3 アクセス権限

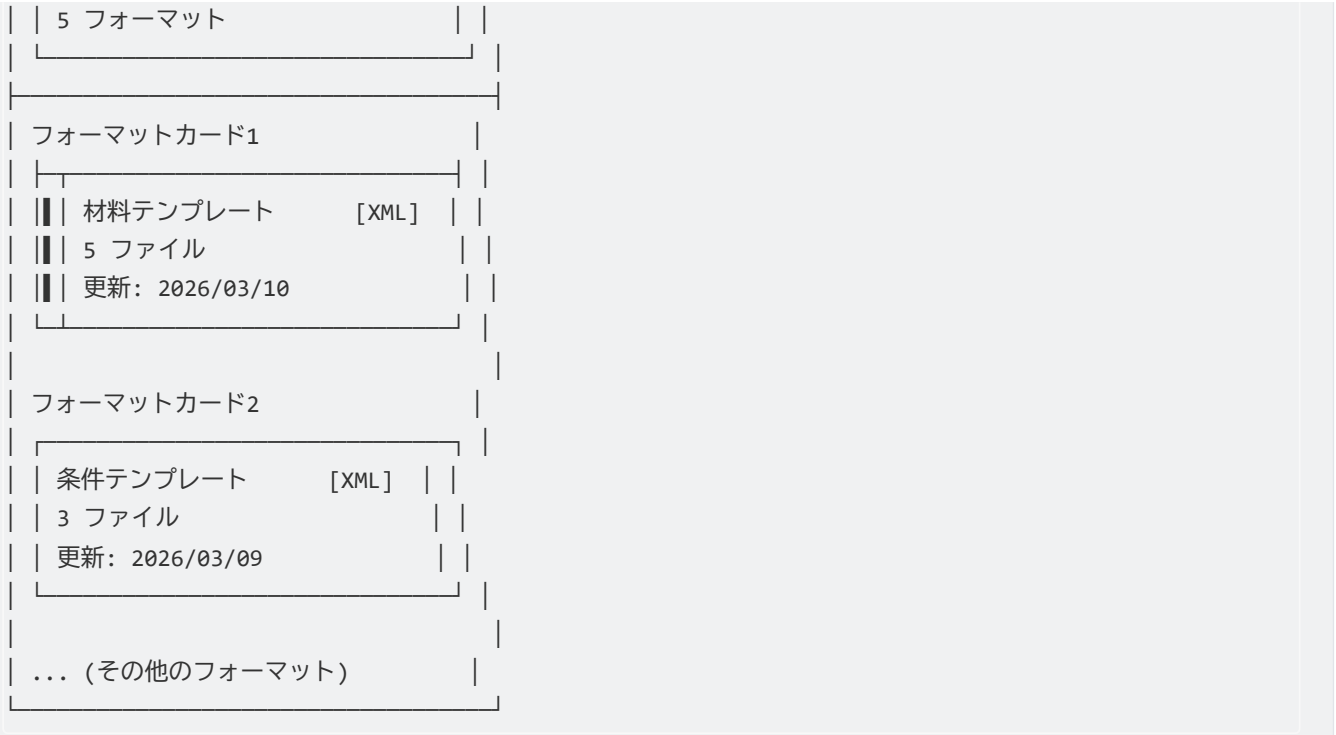
- **必須ロール**: Experiment Admin（実験管理者）
- **認証**: 親コンポーネント（FormatManagement）で認証済み

---

2. 画面構成

2.1 レイアウト構造

```
┌──────────────────────────────────┐
│ ヘッダー                         │
│ ┌──────────────────────────────┐ │
│ │ フォーマット一覧    [ソート]  │ │
```



2.2 主要UI要素

2.2.1 ヘッダー

- **タイトル**（Text strong）:「フォーマット一覧」
- **件数表示**（Text secondary）:「{count} フォーマット」
- **ソートボタン**（Dropdown + Button）: ソート項目選択

2.2.2 フォーマットカード（各フォーマット）

- **フォーマット名**（Text strong）: 選択時は太字
- **ファイルタイプタグ**（Tag）:
  - XML: cyan（水色）
  - MAIML: purple（紫）
- **ファイル数**（Text secondary）:「{count} ファイル」
- **更新日**（Text secondary）:「更新: YYYY/MM/DD」
- **選択インジケーター**: 左側に3pxの青いボーダー、背景色を水色に変更

2.3 インタラクション

2.3.1 フォーマット選択

- **操作**: フォーマットカードをクリック
- **結果**:
  - 選択されたフォーマットがハイライト表示される
  - 親コンポーネントに `onFormatSelect(formatId)` が通知される
  - 右側パネルにファイル一覧が表示される

2.3.2 ソート

- **操作**: ヘッダーのソートボタンをクリック
- **メニュー項目**:
  - フォーマット名
  - カテゴリ
  - 更新日
- **結果**:
  - 親コンポーネントに `onSortChange(sortBy)` が通知される
  - 同じ項目を再度クリックすると昇順⇔降順が切り替わる
  - メニュー項目に現在のソート状態が表示される（↑/↓アイコン）

---

3. 機能仕様

3.1 フォーマット一覧の表示

3.1.1 表示内容

各フォーマットカードには以下の情報が表示されます：

```
<div
  key={format.id}
  onClick={() => onFormatSelect(format.id)}
  style={{
    padding: `${token.paddingSM}px ${token.margin}px`,
    cursor: 'pointer',
    backgroundColor: selectedFormatId === format.id ? token.colorPrimaryBg : 'transparent',
    borderLeft: selectedFormatId === format.id ? `3px solid ${token.colorPrimary}` : '3px
solid transparent',
  }}
>
  <div style={{ display: 'flex', flexDirection: 'column', gap: token.marginXS, width: '100%'
}}>
    <Space size="small">
      <Text
        strong={selectedFormatId === format.id}
        style={{ fontSize: token.fontSize }}
      >
        {format.name}
      </Text>
      <Tag
        color={format.fileType === 'xml' ? 'cyan' : 'purple'}
        style={{ fontSize: token.fontSizeSM, margin: 0 }}
      >
        {format.fileType.toUpperCase()}
      </Tag>
    </Space>
    <Space size="small" wrap>
      <Text type="secondary" style={{ fontSize: token.fontSizeSM }}>
        {format.fileCount} ファイル
      </Text>
    </Space>
    <Text type="secondary" style={{ fontSize: token.fontSizeSM }}>
      更新: {new Date(format.updatedAt).toLocaleDateString('ja-JP')}
    </Text>
  </div>
</div>
```

3.1.2 UI動作

- ユーザー操作: フォーマットカードをクリック
- 画面変化:
  - カードがハイライト表示される（背景色変更 + 左ボーダー表示）
  - フォーマット名が太字になる

- 親コンポーネントへの通知: `onFormatSelect(format.id)`

3.2 カテゴリタグの色分け

3.2.1 カテゴリ色マッピング

```
const getCategoryTagColor = (category: string): string => {
  const colors: Record<string, string> = {
    protocol: 'cyan',      // プロトコル: 水色
    material: 'success',   // 材料: 緑
    measurement: 'blue',   // 測定: 青
    analysis: 'orange',    // 解析: オレンジ
    sample: 'purple',      // 試料: 紫
    device: 'red',         // 装置: 赤
  };
  return colors[category] || 'default';
};
```

3.2.2 カテゴリラベル

```
const getCategoryLabel = (category: string): string => {
  const labels: Record<string, string> = {
    protocol: 'プロトコル',
    material: '材料',
    measurement: '測定',
    analysis: '解析',
    sample: '試料',
    device: '装置',
  };
  return labels[category] || category;
};
```

注: 現在の実装ではカテゴリラベルは表示されていませんが、将来的に使用される可能性があります。

3.3 ソート機能

3.3.1 ソートメニュー

```
const menuItems: MenuProps['items'] = [
  {
    key: 'name',
    label: (
      <Space>
        <span>フォーマット名</span>
        {sortBy === 'name' && (sortOrder === 'asc' ? <SortAscendingOutlined /> :
<SortDescendingOutlined />)}
      </Space>
    ),
    onClick: () => onSortChange('name'),
  },
  {
    key: 'category',
    label: (
      <Space>
        <span>カテゴリ</span>
        {sortBy === 'category' && (sortOrder === 'asc' ? <SortAscendingOutlined /> :
<SortDescendingOutlined />)}
      </Space>
    ),
    onClick: () => onSortChange('category'),
  },
  {
    key: 'updatedAt',
    label: (
      <Space>
        <span>更新日</span>
        {sortBy === 'updatedAt' && (sortOrder === 'asc' ? <SortAscendingOutlined /> :
<SortDescendingOutlined />)}
      </Space>
    ),
    onClick: () => onSortChange('updatedAt'),
  },
];
```

3.3.2 ソート動作

- **ソート項目**:
  1. フォーマット名（name）: 日本語ロケールでの文字列比較
  2. カテゴリ（category）: 文字列比較
  3. 更新日（updatedAt）: 日時比較
- **ソート順序**:

    - 初回クリック: 昇順（asc）
    - 同じ項目を再度クリック: 降順（desc）に切り替え
    - 別の項目をクリック: 昇順（asc）に戻る

- **実装**: 親コンポーネント（FormatManagement）でソート処理を実行

3.3.3 UI動作

- ユーザー操作: ヘッダーのソートボタンをクリック → メニュー項目を選択
- 画面変化:
    - メニューが表示される
    - 現在のソート項目に↑/↓アイコンが表示される
    - フォーマット一覧がソートされる
- 親コンポーネントへの通知: `onSortChange(sortBy)`

3.4 スタイリング

3.4.1 レイアウト

```
<div
  style={{
    height: '100%',
    backgroundColor: token.colorBgContainer,
    borderRight: `1px solid ${token.colorBorderSecondary}`,
    overflowY: 'auto',
  }}
>
```

- **高さ**: 親コンテナの100%（画面全体に伸びる）
- **背景色**: `token.colorBgContainer`（白系）
- **右ボーダー**: 1pxの灰色線
- **オーバーフロー**: 縦スクロール可能

3.4.2 ヘッダー

```
<div
  style={{
    padding: token.margin,
    borderBottom: `1px solid ${token.colorBorderSecondary}`,
  }}
>
```

- **パディング**: `token.margin`
- **下ボーダー**: 1pxの灰色線

3.4.3 選択状態のカード

```
backgroundColor: selectedFormatId === format.id ? token.colorPrimaryBg : 'transparent',
borderLeft: selectedFormatId === format.id ? `3px solid ${token.colorPrimary}` : '3px solid
transparent',
```

- **背景色**: 選択時は水色（`token.colorPrimaryBg`）
- **左ボーダー**: 選択時は3pxの青線（`token.colorPrimary`）

---

4. データ仕様

4.1 Props定義

```
interface FormatFolderTreeProps {
  formats: Format[];                                          // フォーマット一覧
  selectedFormatId: string | null;                            // 選択中のフォーマットID
  onFormatSelect: (formatId: string) => void;                 // フォーマット選択時のコールバック
  sortBy: 'name' | 'category' | 'updatedAt' | null;           // ソート項目
  sortOrder: 'asc' | 'desc';                                  // ソート順序
  onSortChange: (sortBy: 'name' | 'category' | 'updatedAt') => void; // ソート変更時のコールバッ
```

```
ク
}
```

Props説明:

Prop名	型	必須	説明
formats	Format[]	✅	表示するフォーマット一覧
selectedFormatId	string | null	✅	現在選択されているフォーマットのID（未選択時はnull）
onFormatSelect	(formatId: string) => void	✅	フォーマットがクリックされた時に呼び出されるコールバック関数
sortBy	'name' | 'category' | 'updatedAt' | null	✅	現在のソート項目（未ソート時はnull）
sortOrder	'asc' | 'desc'	✅	現在のソート順序（昇順/降順）
onSortChange	(sortBy: string) => void	✅	ソート項目が変更された時に呼び出されるコールバック関数

4.2 Format型定義

```typescript
interface Format {
  id: string;           // フォーマットID
  name: string;         // フォーマット名
  folderName: string;   // フォルダ名
  description: string;  // 説明
  fileCount: number;    // ファイル数
  fileType: 'xml' | 'maiml';  // ファイルタイプ
  category: string;     // カテゴリ
  department: string;   // 部門CD
  createdAt: string;    // 作成日時（ISO 8601形式）
  updatedAt: string;    // 更新日時（ISO 8601形式）
  tags: string[];       // タグ
}
```

Format説明:

フィールド名	型	説明
id	string	フォーマットの一意識別子

name	string	表示用のフォーマット名
folderName	string	S3/Swiftのフォルダ名
description	string	フォーマットの説明
fileCount	number	フォーマット内のファイル数
fileType	'xml' | 'maiml'	フォーマットのファイルタイプ（XML または MAIML）
category	string	カテゴリ（protocol/material/measurement/analysis/sample/device）
department	string	所属部門のコード
createdAt	string	作成日時（ISO 8601形式文字列）
updatedAt	string	最終更新日時（ISO 8601形式文字列）
tags	string[]	フォーマットに関連付けられたタグ

4.3 State定義

このコンポーネントは**ステートレス**で、ローカルStateを持ちません。 全ての状態管理は親コンポーネント（FormatManagement）で行われます。

---

5. API連携

このコンポーネントは直接APIを呼び出しません。 親コンポーネント（FormatManagement）が取得したデータを `formats` Propsとして受け取ります。

---

6. エラーハンドリング

6.1 エラーケース

このコンポーネントは表示専用のため、独自のエラーハンドリングはありません。

以下のケースは親コンポーネントで処理されます：

- フォーマット一覧取得失敗
- データの不整合（nullやundefined）

6.2 表示の制約

6.2.1 空の一覧

```
{formats.map((format) => (
  // ... フォーマットカードの表示
))}
```

- `formats` が空配列の場合、カードは何も表示されません
- ヘッダーには「0 フォーマット」と表示されます

6.2.2 日付フォーマットエラー

```
更新: {new Date(format.updatedAt).toLocaleDateString('ja-JP')}
```

- `updatedAt` が不正な日付文字列の場合、「Invalid Date」と表示される可能性があります
- **推奨**: 親コンポーネントでデータバリデーションを実施

---

7. 実装ファイル

7.1 ファイルパス

- **メインファイル**: `frontend/src/components/project/FormatFolderTree.tsx`

7.2 関連ファイル

- **親コンポーネント**: `frontend/src/components/project/FormatManagement.tsx`
- **型定義**: このファイル内で定義（Format型）

7.3 依存ライブラリ

- **Ant Design**:
  - Typography (Text)
  - Tag
  - Dropdown
  - Space
  - Button
  - theme
- **Ant Design Icons**:
  - SortAscendingOutlined
  - SortDescendingOutlined

7.4 親子関係

```
FormatManagement (親)
  ↓ Props: formats, selectedFormatId, onFormatSelect, sortBy, sortOrder, onSortChange
FormatFolderTree (このコンポーネント)
```

親コンポーネントから受け取るデータ:

- フォーマット一覧（フィルタリング・ソート済み）
- 選択状態
- ソート設定

親コンポーネントへの通知:

- フォーマット選択イベント
- ソート変更イベント

---

8. UI/UXの考慮事項

8.1 レスポンシブデザイン

- **幅**: 親コンポーネント（FormatManagement）で33%に固定
- **最小幅**: 350px（親コンポーネントで設定）
- **最大幅**: 500px（親コンポーネントで設定）
- **高さ**: 親コンテナの100%

8.2 スクロール

- **縦スクロール**: `overflowY: 'auto'` で自動表示
- フォーマット数が多い場合、スクロールバーが表示される
- ヘッダーは固定（スクロール対象外）

8.3 視覚的フィードバック

8.3.1 ホバー効果

```
cursor: 'pointer'
```

- マウスカーソルがポインターに変わり、クリック可能であることを示す

8.3.2 選択状態

- **背景色**: 水色（ `token.colorPrimaryBg` ）
- **左ボーダー**: 3pxの青線（ `token.colorPrimary` ）
- **フォント**: 太字（ `strong={true}` ）

8.3.3 ファイルタイプの識別

- **XMLタグ**: cyan（水色）
- **MAIMLタグ**: purple（紫）

8.4 アクセシビリティ

- **クリック領域**: カード全体がクリック可能
- **視覚的な階層**: 明確な選択状態の表示
- **テキストの可読性**: 適切なフォントサイズとコントラスト

---

9. 変更履歴

日付	変更内容	担当者
2026-03-11	実装ベースで仕様書を作成（Phase 4: フォーマット管理画面）	Cline